

TASK 2 DOCUMENTATION

Project Title:

Exploratory Data Analysis (EDA) on Dataset for Data Analytics

Abstract

Exploratory Data Analysis (EDA) is an important step in the data analytics process that helps analysts understand the structure, patterns, and characteristics of a dataset. This project focuses on analyzing the Dataset for Data Analytics using descriptive statistics and trend analysis. The analysis was performed to identify patterns, detect outliers, analyze sales trends, and derive meaningful insights from the dataset.

1. Introduction

Data analysis begins with understanding the dataset before applying advanced analytical or machine learning techniques. Exploratory Data Analysis (EDA) is the process of examining data through statistical summaries and analytical techniques to uncover hidden patterns, trends, and relationships.

This project involves performing EDA on the Dataset for Data Analytics using Python libraries such as Pandas. The analysis helps in understanding the characteristics of the dataset and supports informed decision-making.

2. Objectives

- To understand the structure and characteristics of the dataset.
- To calculate descriptive statistical measures.
- To identify trends and patterns in the dataset.
- To detect potential outliers.
- To analyze sales and order trends.
- To summarize key observations and insights.

3. Problem Statement

Organizations collect large amounts of data that often contain valuable information. However, without proper analysis, it is difficult to understand trends, patterns, and relationships within the data. The objective of this project is to perform Exploratory Data Analysis (EDA) to extract meaningful insights and support data-driven decision-making.

4. Dataset Description

Dataset Name: cleaned_dataset

The cleaned dataset obtained after the Data Cleaning and Preprocessing phase was used for Exploratory Data Analysis. The dataset was analyzed to understand its statistical properties, identify trends, detect outliers, and derive meaningful insights.

Dataset Information

- Dataset Name: cleaned_dataset
- File Format: Excel (.xlsx)

- Domain: Data Analytics
- Purpose: Exploratory Data Analysis

5. Tools and Technologies Used

- Jupyter Notebook
- Pandas
- Microsoft Excel

6. Methodology

Step 1: Data Loading

The dataset was imported into Jupyter Notebook using Pandas.

```
[1]: # Import required libraries
import pandas as pd

# Load Dataset
df = pd.read_excel('C:/Users/LENOVO/Downloads/cleaned_dataset.xlsx')

# Display dataset
df
```

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	Ref
0	ORD200000	2023-01-04	C72849	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	7	SAVE10	
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186779	3	SAVE10	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	8	FREESHIP	
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK62788070	5	SAVE10	
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
1195	ORD201195	2024-06-20	C21126	Desk	1	107.04	392 Main St	Credit Card	Cancelled	TRK38009181	6	FREESHIP	
1196	ORD201196	2024-03-04	C20095	Monitor	2	662.53	778 Main St	Online	Cancelled	TRK69207593	5	No Coupon	
1197	ORD201197	2023-07-13	C79674	Tablet	2	436.84	275 Main St	Online	Delivered	TRK38039356	2	FREESHIP	
1198	ORD201198	2024-08-22	C64753	Chair	4	262.52	509 Main St	Debit Card	Cancelled	TRK71683331	4	WINTER15	
1199	ORD201199	2023-06-11	C57502	Tablet	4	580.58	201 Main St	Gift Card	Returned	TRK51116746	6	SAVE10	

1200 rows x 14 columns

Step 2: Data Inspection

The dataset structure was examined using:

- Shape of dataset and information of dataset

```
[26]: # Display dataset information including
df.info()

#Dataset shape
print("Shape of dataset: ",df.shape)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 14 columns):
#   Column             Non-Null Count  Dtype
---  ---
0   OrderID             1200 non-null   str
1   Date                1200 non-null   datetime64[us]
2   CustomerID          1200 non-null   str
3   Product             1200 non-null   str
4   Quantity            1200 non-null   int64
5   UnitPrice           1200 non-null   float64
6   ShippingAddress     1200 non-null   str
7   PaymentMethod       1200 non-null   str
8   OrderStatus         1200 non-null   str
9   TrackingNumber      1200 non-null   str
10  ItemsInCart         1200 non-null   int64
11  CouponCode          1200 non-null   str
12  ReferralSource       1200 non-null   str
13  TotalPrice          1200 non-null   float64
dtypes: datetime64[us](1), float64(2), int64(2), str(9)
memory usage: 131.4 KB
Shape of dataset: (1200, 14)
```

7. Exploratory Data Analysis Tasks Performed

Descriptive Statistics

- Calculated mean values.
- Calculated median values.
- Analyzed minimum and maximum values.
- Examined count and spread of data.

```
[27]: # Generate descriptive statistics
print(df.describe())
```

	Date	Quantity	UnitPrice	ItemsInCart	TotalPrice
count	1200	1200.000000	1200.000000	1200.000000	1200.000000
mean	2024-03-22 16:58:48	2.945833	356.412750	5.485000	1053.968300
min	2023-01-01 00:00:00	1.000000	11.390000	1.000000	11.390000
25%	2023-08-03 18:00:00	2.000000	186.062500	4.000000	410.520000
50%	2024-03-23 00:00:00	3.000000	364.210000	5.000000	823.615000
75%	2024-11-08 12:00:00	4.000000	521.570000	7.000000	1578.475000
max	2025-06-30 00:00:00	5.000000	699.930000	10.000000	3456.400000
std	NaN	1.407557	197.177146	2.281983	819.856558

Product Sales Analysis

- Calculated total quantity sold for each product.
- Identified the most sold and least sold products.

```
[18]: # Calculate total quantity sold for each product
product_sales = df.groupby('Product')['Quantity'].sum().reset_index()

# Display product-wise sales quantity
print(product_sales)
```

	Product	Quantity
0	Chair	562
1	Desk	508
2	Laptop	535
3	Monitor	480
4	Phone	411
5	Printer	542
6	Tablet	497

Daily Sales Analysis

- Calculated total sales for each day.
- Analyzed daily sales trends.

```
[19]: # Calculate daily sales trend
daily_sales = df.groupby('Date')['TotalPrice'].sum()

# Display daily sales trend
print("Daily Sales Trend:\n",daily_sales)
```

```
Daily Sales Trend:
Date
2023-01-01    2021.11
2023-01-02    1323.55
2023-01-03     897.70
2023-01-04    3629.54
2023-01-05    3642.30
...
2025-06-24     6358.22
2025-06-25     391.83
2025-06-27     1764.15
2025-06-28     3894.68
2025-06-30      505.47
Name: TotalPrice, Length: 671, dtype: float64
```

Monthly Sales Analysis

- Calculated total sales for each month.
- Identified months with highest and lowest sales.

```
[15]: # Extract month from date
df['Month'] = df['Date'].dt.to_period('M')

# Calculate monthly sales
monthly_sales = df.groupby('Month')['TotalPrice'].sum()

# Display monthly sales
print(monthly_sales)
```

Month	TotalPrice
2023-01	56685.75
2023-02	48117.66
2023-03	48699.37
2023-04	27751.71
2023-05	63836.84
2023-06	49598.19
2023-07	42828.66
2023-08	54352.14
2023-09	29526.67
2023-10	52697.85
2023-11	43879.67
2023-12	43754.73
2024-01	38528.08
2024-02	36909.57
2024-03	36838.90
2024-04	49613.14
2024-05	27909.11
2024-06	68068.54
2024-07	42963.98
2024-08	31991.07
2024-09	39794.98
2024-10	37225.97
2024-11	32413.76
2024-12	38785.77
2025-01	29899.48
2025-02	35317.55
2025-03	39206.66
2025-04	31821.20
2025-05	43396.64
2025-06	53047.40

Freq: M, Name: TotalPrice, dtype: float64

Monthly Order Analysis

- Counted orders placed each month.
- Analyzed monthly order trends.

```
[14]: # Count orders per month
monthly_orders = df.groupby('Month')['OrderID'].count()

# Display monthly orders
print(monthly_orders)
```

Month	OrderID
2023-01	47
2023-02	37
2023-03	43
2023-04	31
2023-05	49
2023-06	45
2023-07	44
2023-08	51
2023-09	29
2023-10	47
2023-11	41
2023-12	46
2024-01	32
2024-02	32
2024-03	36
2024-04	58
2024-05	34
2024-06	53
2024-07	43
2024-08	28
2024-09	44
2024-10	31
2024-11	35
2024-12	41
2025-01	27
2025-02	37
2025-03	49
2025-04	32
2025-05	37
2025-06	49

Freq: M, Name: OrderID, dtype: int64

Order Status Analysis

- Analyzed frequency of different order statuses.
- Identified the most common order status.

```
[21]: # Order Status Distribution
order_status = df['OrderStatus'].value_counts()

print(order_status)

OrderStatus
Cancelled    250
Returned    247
Pending      237
Shipped      235
Delivered    231
Name: count, dtype: int64
```

Payment Method Analysis

- Analyzed payment method usage.
- Identified the most preferred payment method.

```
[20]: # Payment Method Distribution
payment_method = df['PaymentMethod'].value_counts()

print(payment_method)

PaymentMethod
Online      258
Cash        246
Credit Card 234
Debit Card  232
Gift Card   230
Name: count, dtype: int64
```

Outlier Detection

- Detected unusual values.
- Analyzed their impact on the dataset.

```
[15]: # Outlier Detection for TotalPrice using IQR Method

# Calculate first quartile (25%)
Q1 = df['TotalPrice'].quantile(0.25)

# Calculate third quartile (75%)
Q3 = df['TotalPrice'].quantile(0.75)

# Calculate Interquartile Range
IQR = Q3 - Q1

# Calculate lower and upper limits for outlier detection
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

# Identify outlier records
outliers = df[
    (df['TotalPrice'] < lower_limit) |
    (df['TotalPrice'] > upper_limit)
]

# Display outlier information
print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)
print("Number of Outliers:", len(outliers))

# Display outlier rows
print(outliers[['Product', 'Quantity', 'TotalPrice']])

Lower Limit: -1341.4125
Upper Limit: 3338.4875
Number of Outliers: 8
Product Quantity TotalPrice
187 Printer 5 3353.75
326 Laptop 5 3352.49
328 Tablet 5 3370.28
469 Chair 5 3384.98
632 Laptop 5 3398.88
789 Tablet 5 3456.48
1865 Printer 5 3334.00
1122 Monitor 5 3398.95
```

8. Results

The Exploratory Data Analysis revealed:

- Statistical characteristics of the dataset.
- Product sales trends.
- Daily sales patterns.
- Monthly sales trends.
- Monthly order trends.
- Order status distribution.
- Payment method preferences.

- Presence of outliers.
- Key insights useful for decision-making.

9. Key Observations

- Product sales varied significantly across products.
- Daily and monthly sales trends were identified.
- Order volumes differed across months.
- Certain payment methods were used more frequently than others.
- Statistical summaries provided valuable insights into dataset behavior.
- Potential outliers were identified in numerical columns.

10. Challenges Faced

- Identifying significant outliers.
- Analyzing large volumes of data.
- Interpreting sales and order trends accurately.
- Extracting meaningful insights from the dataset.

11. Conclusion

The Exploratory Data Analysis project successfully examined the Dataset for Data Analytics and provided valuable insights into its structure, sales trends, order patterns, payment preferences, and overall behavior. Statistical analysis and trend analysis helped uncover important patterns and identify outliers. The findings can support further analysis, reporting, and business decision-making activities.

12. References

1. Pandas Documentation
2. Jupyter Notebook Documentation
3. Microsoft Excel Documentation
4. DecodeLabs Industrial Training Kit